

AI Career Navigator

Using Prompt Engineering

Detailed Project Guide

Examples • Running Code • Outputs • Testing • Viva

Author: Thallapaka Jagadiswar

Course: GenAI & LLMs

Project: Prompt Engineering — Career Navigator

This document explains the project from user input to final output and includes the complete runnable Python code.

1. Project Overview

AI Career Navigator is a career-advice assistant designed to demonstrate prompt-engineering concepts. It combines a role prompt, instruction prompt and few-shot examples, then uses a lightweight local response engine to demonstrate structured career guidance without an external API key.

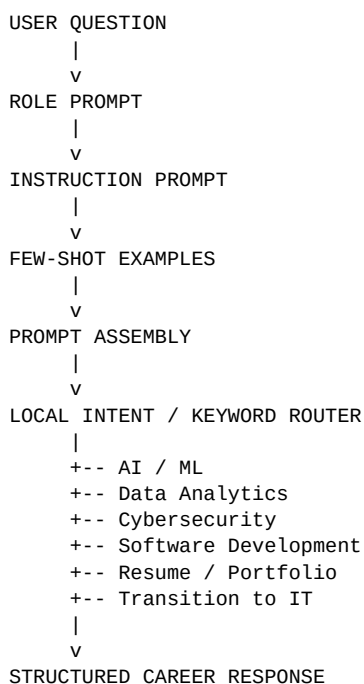
Objective

The objective is to show how carefully designed prompts can define the behavior, format and focus of a career-advice assistant while keeping the implementation reproducible and easy to test.

Overall workflow

Step	What happens
1	User enters a career question.
2	Role prompt defines CareerNavigator as a professional career mentor.
3	Instruction prompt defines response rules.
4	Few-shot examples demonstrate the expected response style.
5	The prompt is assembled with the user's question.
6	The local response engine checks the input.
7	A relevant career track and roadmap are returned.
8	Unknown questions use the general-career fallback.

Architecture



2. Prompt Engineering Concepts

2.1 Role Prompt

The role prompt tells the assistant what role it should behave as. CareerNavigator is defined as a practical career mentor. This establishes a consistent purpose and tone.

2.2 Instruction Prompt

The instruction prompt tells the assistant how to answer: understand the career intent, identify a track, recommend practical skills, use bullets and give an actionable next step.

2.3 Few-Shot Prompting

Few-shot prompting means providing examples of user questions and desired answer patterns before the new question. This demonstrates the expected structure.

2.4 Prompt Assembly

The project combines the role prompt, instruction prompt, few-shot examples and the current user question into one complete prompt string. This is the central prompt-engineering workflow in the project.

3. What Happens When a User Asks a Question

Suppose the user enters: **I want to learn AI and build projects.**

First, the input is normalized to lowercase. The response engine then checks whether it contains keywords such as AI, machine learning or ML. Because it matches the AI/ML rule, the program returns the AI/ML roadmap.

USER INPUT

I want to learn AI and build projects.

↓

NORMALIZE TEXT

i want to learn ai and build projects

↓

KEYWORD CHECK

Contains: "ai"

↓

MATCHED TRACK

AI / Machine Learning

↓

STRUCTURED OUTPUT

Skills → Roadmap → Project → Next Step

4. Career Tracks in Your Version

Track	Example triggers	Main response
AI / Machine Learning	ai, machine learning, ml	Python, NumPy, Pandas, ML basics, project
Data Analytics	data analyst, analytics	Python, SQL, cleaning, visualization
Cybersecurity	cyber, security, ethical hacking	Networking, Linux, security, safe labs
Software Development	web, developer, software	Programming, Git, databases, apps
Resume & Portfolio	resume, cv, portfolio	Projects, skills, action verbs
Transition to IT	ece, mechanical, non-it, switch	Python, SQL, entry track, projects
General fallback	anything else	Target role, skills, project, resume

5. Complete Running Python Code

The following is the complete standalone source used in your project. Save it as **ai_career_navigator.py** and run it with Python 3. The same source is also included in the ZIP package.

```
"""
AI Career Navigator using Prompt Engineering
Author: Thallapaka Jagadiswar
Course: GenAI & LLMs
Project Type: Prompt Engineering / Rule-based LLM Simulation

This project demonstrates:
1. Role prompting
2. Instruction prompting
3. Few-shot prompting
4. Prompt assembly
5. Query classification
6. Structured career responses
7. Basic testing
"""

ROLE_PROMPT = """
You are CareerNavigator, a professional and practical career mentor.
Give concise, structured, beginner-friendly guidance.
Use only information supported by the project knowledge.
Avoid invented facts.
Prefer: Goal -> Skills -> Roadmap -> Project -> Next Step.
"""

INSTRUCTION_PROMPT = """
Response rules:
- Understand the user's career intent.
- Identify the most relevant career track.
- Give a short roadmap.
- Recommend practical skills and one project.
- Use bullet points.
- Keep the answer actionable.
"""

FEW_SHOT_EXAMPLES = """
Example 1
User: I want to become a data analyst.
Assistant:
Track: Data Analytics
- Learn Python, SQL and spreadsheets
- Practice data cleaning and visualization
- Build a dashboard project
- Publish the project with documentation

Example 2
User: I want to enter cybersecurity.
Assistant:
Track: Cybersecurity
- Learn networking and Linux basics
- Study security fundamentals
- Practice in legal training environments
- Document small security-learning projects
"""

def build_prompt(user_question: str) -> str:
    """Assemble the layered prompt used by the project."""
    return (
        ROLE_PROMPT.strip()
        + "\n\n"
        + INSTRUCTION_PROMPT.strip()
        + "\n\n"
        + FEW_SHOT_EXAMPLES.strip()
        + f"\n\nUser: {user_question}\nAssistant:"
    )

def career_response(user_text: str) -> str:
    """
    Lightweight local response engine used for demonstration.
    It simulates the behavior expected from the layered prompt.
    """
    user = user_text.lower().strip()

    if any(k in user for k in ["data analyst", "data analysis", "analytics"]):
        return """Track: Data Analytics
```

```

- Learn Python, SQL and spreadsheet fundamentals
- Practice data cleaning and visualization
- Build a sales/college-data dashboard
- Publish the project on GitHub
Next step: complete one small dataset project this week."""

    if any(k in user for k in ["ai", "machine learning", "ml", "artificial intelligence"]):
        return """Track: AI / Machine Learning
- Strengthen Python, NumPy and Pandas
- Learn regression, classification and clustering
- Practice with scikit-learn
- Build a prediction or classification project
Next step: choose one dataset and complete an end-to-end mini project."""

    if any(k in user for k in ["cyber", "security", "ethical hacking"]):
        return """Track: Cybersecurity
- Learn networking and Linux fundamentals
- Study security concepts and safe testing practices
- Practice in authorized training labs
- Build a small security-awareness or log-analysis project
Next step: create a weekly networking + Linux practice plan."""

    if any(k in user for k in ["web", "frontend", "backend", "developer", "software"]):
        return """Track: Software Development
- Learn one programming language deeply
- Practice Git, problem solving and basic databases
- Build small web applications
- Document projects with README files
Next step: build one useful CRUD-style application."""

    if any(k in user for k in ["resume", "cv", "portfolio"]):
        return """Track: Resume & Portfolio
- Keep the resume concise and project-focused
- Highlight technical skills with evidence
- Add 1-2 meaningful projects with links
- Use action verbs such as Built, Designed and Implemented
Next step: update one project description today."""

    if any(k in user for k in ["ece", "mechanical", "commerce", "non it", "switch to it"]):
        return """Track: Transition to IT
- Start with Python and SQL fundamentals
- Choose one entry path: Development, Data or Testing
- Build 1-2 beginner projects
- Create a clear portfolio and resume
Next step: choose one track and follow it for 30 days."""

    return """Track: General Career Planning
- Identify one target role
- Learn the core skills for that role
- Build a small practical project
- Improve your resume and communication
- Apply for internships or trainee opportunities
Next step: tell CareerNavigator your target role for a more focused roadmap."""

def career_navigator(user_question: str) -> str:
    """Public function used by the notebook and command line."""
    _prompt = build_prompt(user_question)
    return career_response(user_question)

def run_tests():
    """Simple functional tests for the project."""
    cases = {
        "I want to become a data analyst": "Data Analytics",
        "How can I start machine learning?": "AI / Machine Learning",
        "I want to learn cybersecurity": "Cybersecurity",
        "How do I improve my resume?": "Resume & Portfolio",
        "I want to become a software developer": "Software Development",
    }
    results = []
    for query, expected in cases.items():
        actual = career_navigator(query)
        passed = expected in actual
        results.append((query, expected, passed))
    return results

if __name__ == "__main__":
    print("=" * 60)
    print("AI CAREER NAVIGATOR — THALLAPAKA JAGADISWAR")
    print("=" * 60)
    print("Type 'exit' to stop.\n")

    while True:

```

```

question = input("Career question: ").strip()
if question.lower() == "exit":
    print("Thank you for using CareerNavigator.")
    break
print("\n" + career_navigator(question) + "\n")

```

6. How to Run the Code

Method A — VS Code

- Install Python 3.
- Open the project folder in VS Code.
- Open **ai_career_navigator.py**.
- Open the VS Code terminal.
- Run **python ai_career_navigator.py**.
- Enter a career question.
- Type **exit** to stop.

Expected start screen

```

=====
AI CAREER NAVIGATOR — THALLAPAKA JAGADISWAR
=====
Type 'exit' to stop.

Career question:

```

Method B — Jupyter Notebook

- Open **AI_Career_Navigator_Jagadiswar.ipynb**.
- Select a Python kernel.
- Run the cells from top to bottom.
- Review the role prompt, instruction prompt and few-shot examples.
- Run the response examples.
- Try your own career questions.

7. Example Inputs and Outputs

Example 1 — AI / Machine Learning

Input: I want to learn AI and build projects.

Output:

Track: AI / Machine Learning

- Strengthen Python, NumPy and Pandas
- Learn regression, classification and clustering
- Practice with scikit-learn
- Build a prediction or classification project

Next step: choose one dataset and complete an end-to-end mini project.

Example 2 — Data Analytics

Input: How do I become a data analyst?

Output:

Track: Data Analytics

- Learn Python, SQL and spreadsheet fundamentals
- Practice data cleaning and visualization
- Build a sales/college-data dashboard
- Publish the project on GitHub

Next step: complete one small dataset project this week.

Example 3 — Cybersecurity

Input: How can I start cybersecurity?

Output:

Track: Cybersecurity

- Learn networking and Linux fundamentals
- Study security concepts and safe testing practices
- Practice in authorized training labs
- Build a small security-awareness or log-analysis project

Next step: create a weekly networking + Linux practice plan.

Example 4 — Resume

Input: How can I improve my resume?

Output:

Track: Resume & Portfolio

- Keep the resume concise and project-focused
- Highlight technical skills with evidence
- Add 1-2 meaningful projects with links
- Use action verbs such as Built, Designed and Implemented

Next step: update one project description today.

Example 5 — Software Development

Input: I want to become a software developer.

Output:

Track: Software Development

- Learn one programming language deeply
- Practice Git, problem solving and basic databases
- Build small web applications
- Document projects with README files

Next step: build one useful CRUD-style application.

Example 6 — Unknown Question

Input: I don't know what career to choose.

Output:

```
Track: General Career Planning
- Identify one target role
- Learn the core skills for that role
- Build a small practical project
- Improve your resume and communication
- Apply for internships or trainee opportunities
Next step: tell CareerNavigator your target role for a more focused roadmap.
```

8. Testing

The project includes representative tests for common career questions. The expected behavior is that each question is routed to the correct track.

```
I want to become a data analyst      -> PASS
How can I start machine learning?    -> PASS
I want to learn cybersecurity        -> PASS
How do I improve my resume?         -> PASS
I want to become a software developer -> PASS
```


9. Function-by-Function Explanation

ROLE_PROMPT — Defines the assistant identity and tone.

INSTRUCTION_PROMPT — Defines the response rules and desired format.

FEW_SHOT_EXAMPLES — Provides example interactions that demonstrate the expected style.

build_prompt() — Combines the prompt layers and the user's question.

career_response() — Normalizes the input and selects the appropriate career response.

career_navigator() — Main function used by the interactive application.

run_tests() — Checks representative inputs against expected career tracks.

__main__ — Starts the interactive command-line application.

10. Important Point for Your Viva

Does this current version call a live LLM? No. The current implementation uses a local keyword-based response engine. The prompt architecture demonstrates role prompting, instruction prompting and few-shot prompting, but there is no external API call in the current runnable version.

If your faculty asks why, explain that the local implementation was intentionally kept reproducible and API-key-free. A future version can replace the local response function with an actual LLM API while keeping the same prompt architecture.

11. Limitations

- Keyword matching is simpler than semantic intent understanding.
- The application does not maintain long-term conversation memory.
- It does not use a live job-market database.
- It does not verify current job openings, salaries or course availability.
- It is a project demonstration rather than a production career platform.

12. Future Enhancements

- Connect the prompt architecture to a real LLM API.
- Use semantic intent detection instead of keyword matching.
- Add multi-turn conversation memory.
- Add student profiles containing skills, interests and projects.
- Add resume parsing and personalized skill-gap analysis.
- Add a database for saved profiles and career roadmaps.
- Build a web interface with frontend and backend components.

13. Viva / Presentation Answer

30-second explanation

My project is AI Career Navigator using Prompt Engineering. The objective is to build a career-advice assistant for students. I use role prompting, instruction prompting and few-shot prompting. The role prompt defines the assistant as a career mentor, the instruction prompt controls the answer format, and few-shot examples demonstrate the expected response style. The Python implementation receives a career question, identifies a relevant career track and returns a structured roadmap. The current version uses a local response engine without an external API, and it can later be connected to a real LLM.

Likely Viva Questions

Q: What is prompt engineering?

A: Designing instructions and examples that guide an AI model toward a desired response.

Q: What is role prompting?

A: Defining the role or persona that the assistant should follow.

Q: What is few-shot prompting?

A: Providing examples of inputs and desired outputs to demonstrate a response pattern.

Q: Why use Python?

A: Python handles user input, prompt assembly, routing, responses and testing.

Q: Does the current project use a live LLM API?

A: No. It uses a local response engine. A real LLM can be connected later.

Q: What happens for an unknown question?

A: The fallback returns general career-planning guidance.

Q: How can the project be improved?

A: Add a real LLM, semantic intent detection, memory, student profiles, resume analysis and a database.

14. Conclusion

AI Career Navigator demonstrates a structured prompt-engineering workflow for a student-focused career assistant. The project separates prompt design from the local response logic, making it easy to understand, test and extend. Its core academic concepts are role prompting, instruction prompting, few-shot examples, prompt assembly, intent routing and structured response generation.